

PROYECTO MODULAR – MOJO

Documentación técnica del proyecto

Autores:

David Vargas Ramírez

Juan José Morales Benítez

Corporación Universitaria Minuto de Dios

Facultad de Ingeniería

Programa Ingeniería de Sistemas

Curso: Bases de Datos Masivas

Fecha: 24 de abril de 2026

RESUMEN

El presente documento describe la arquitectura, el flujo de datos y los procesos de extracción, transformación y carga.

Palabras clave: bases de datos masivas, MongoDB, SQLite, ETL, Docker, Mojolicious.

1. Introducción

El Proyecto Modular – Mojo tiene como objetivo demostrar un flujo completo de tratamiento de datos masivos con tecnologías modernas.

El proyecto se desarrolla en el marco del curso Bases de Datos Masivas del programa de Ingeniería de Sistemas, de la Corporación Universitaria Minuto de Dios.

2. Arquitectura general del sistema

La solución se despliega mediante Docker y docker-compose, definiendo una arquitectura de cuatro contenedores:

1) Tres contenedores con instancias de MongoDB, cada uno dedicado a un dominio de datos específico:

- a) `mongodb_personas`: almacena documentos de la colección `personas`.
- b) `mongodb_articulos`: almacena documentos de la colección `articulos`.
- c) `mongodb_ventas`: almacena documentos de la colección `ventas`.

2) Un contenedor de aplicación que incluye:

- a) Un servicio web desarrollado con el framework Mojolicious.
- b) Una base de datos relacional SQLite (`almacen.sqlite`) montada como volumen para persistencia.

Desde el cuaderno de Jupyter, se utiliza la librería `python_on_whales` para ejecutar las siguientes operaciones:

- Construcción de imágenes: `docker.compose.build()`.
- Levantamiento de servicios: `docker.compose.up(detach=True)`.

Estas acciones permiten instanciar la infraestructura necesaria para la carga de datos en MongoDB, la exposición

3. Carga inicial de datos en MongoDB

Los datos de origen se encuentran en archivos JSON: `personas.json`, `articulos.json` y `ventas.json`. Cada archivo con

A modo de ejemplo, se emplean comandos equivalentes a los siguientes:

- Importación de personas:

```
docker exec mongodb_personas mongoimport --username root --password root --authenticationDatabase admin
```

- Importación de artículos:

```
docker exec mongodb_articulos mongoimport --username root --password root --authenticationDatabase admin
```

- Importación de ventas:

```
docker exec mongodb_ventas mongoimport --username root --password root --authenticationDatabase admin --
```

Al finalizar el proceso, cada colección (personas, articulos y ventas) contiene aproximadamente 10.000 documentos.

4. Servicio web en Mojolicious

El contenedor de aplicación incluye un servicio HTTP implementado con el framework Mojolicious. Este servicio

- Endpoint de inicialización:

- GET /load_data: verifica el estado del sistema y confirma que las tres bases de datos MongoDB están cargadas.

- Endpoints de consulta de datos en formato JSON:

- GET /mongopersonas y /mongopersonas/all para acceder a la colección personas.

- GET /mongoarticulos y /mongoarticulos/all para acceder a la colección articulos.

- GET /mongoventas y /mongoventas/all para acceder a la colección ventas.

Desde el cuaderno de Jupyter, dichos endpoints se consumen mediante utilidades como curl y la librería requests.

5. Base de datos relacional SQLite y esquema lógico

La base de datos relacional se implementa en un archivo `almacen.sqlite`, montado como volumen dentro del contenedor.

El esquema relacional mínimo definido para el proyecto está compuesto por las tablas siguientes:

- Tabla personas:

- `numeroDocumento` (clave primaria).
- `nombres`.
- `primerApellido`.
- `segundoApellido`.
- `email`.
- `telefono`.
- `direccion`.
- `fechaNacimiento`.

- Tabla articulos:

- `idArticulo` (clave primaria).
- `nombreArticulo`.
- `cantidadArticulo`.
- `precioArticulo`.

- Tabla ventas:

- `idVenta` (clave primaria).
- `idComprador` (clave foránea hacia `personas.numeroDocumento`).
- `idArticulo` (clave foránea hacia `articulos.idArticulo`).
- `cantidadProductos`.
- `precioTotal`.

La integridad referencial se garantiza declarando claves foráneas en la tabla ventas hacia las dimensiones personas y artículos.

6. Función de limpieza de valores numéricos

Los datos provenientes de MongoDB contienen, en algunos campos, caracteres no numéricos mezclados con dígitos.

El comportamiento general de esta función es el siguiente:

- 1) Si el valor recibido es una cadena de texto, se aplica una expresión regular que elimina todo carácter que no sea un dígito.
- 2) Se intenta convertir el resultado a tipo numérico (entero o flotante).
- 3) En caso de error de conversión, la función devuelve el valor 0.

Esta lógica se reutiliza en los tres procesos ETL (personas, artículos y ventas) para garantizar que los campos numéricos estén correctamente formateados.

7. Proceso ETL de artículos

El primer proceso ETL se enfoca en la migración de la colección `base_articulos.articulos` hacia la tabla `articulos`.

En la fase de extracción, el notebook realiza una petición GET al endpoint `http://localhost:8000/mongoarticulos/articulos`.

Durante la fase de transformación, para cada documento se aplican las siguientes reglas:

- `idArticulo_limpio = limpiar_valor_numerico(idArticulo)`.
- `cantidad_limpia = limpiar_valor_numerico(cantidadArticulo)`.
- `precio_limpio = limpiar_valor_numerico(precioArticulo)`.

- Si nombreArticulo no está presente, se asigna el valor por defecto "Sin nombre".

Finalmente, en la fase de carga se abre una conexión a almacen.sqlite, se desactivan temporalmente las claves for

```
INSERT OR REPLACE INTO articulos(idArticulo, nombreArticulo, cantidadArticulo, precioArticulo) VALUE
```

Como resultado, se migran los 10.000 artículos desde MongoDB a la base relacional, con cantidades y precios n

8. Proceso ETL de personas

El segundo proceso ETL se centra en la colección base_personas.personas y la tabla personas. Su objetivo es nor

En la extracción, se invoca el endpoint <http://localhost:8000/mongopersonas/all>, obteniendo la lista de document

- numeroDocumento_limpio = limpiar_valor_numerico(numeroDocumento).
- telefono_limpio = limpiar_valor_numerico(telefono).
- Se ignora el campo _id generado por MongoDB y no se utiliza en el modelo relacional.

Posteriormente, se insertan o reemplazan los registros en la tabla personas mediante la sentencia:

```
INSERT OR REPLACE INTO personas(numeroDocumento, nombres, primerApellido, segundoApellido, email,
```

El resultado es un conjunto de 10.000 personas con documentos y teléfonos estandarizados como números, listos

9. Proceso ETL de ventas

El tercer proceso ETL es el más estricto, ya que implementa reglas de integridad referencial y descarta aquellas v

En la fase de extracción, se obtiene la lista de ventas desde el endpoint `http://localhost:8000/mongoventas/all`. A

1) Se borra el contenido existente de la tabla ventas (`DELETE FROM ventas`) para asegurar una carga limpia.

2) Se cargan en memoria los conjuntos de identificadores válidos:

- `ids_personas = {numeroDocumento de personas}`.

- `ids_articulos = {idArticulo de articulos}`.

Para cada documento de venta se aplican dos filtros principales antes de la inserción:

- Filtro 1 (tipado): `idComprador` e `idArticulo` deben ser convertibles a enteros. Si la conversión falla, la venta se o

- Filtro 2 (referencial): `idComprador` debe existir en `ids_personas` e `idArticulo` en `ids_articulos`. Si alguno no exis

Tras superar dichos filtros, se limpian los campos `cantidadProductos` y `precioTotal` con la función `limpiar_valor`

`INSERT OR REPLACE INTO ventas(idVenta, idComprador, idArticulo, cantidadProductos, precioTotal).`

Cuando se presentan conflictos de integridad en SQLite, se capturan las excepciones correspondientes y se omite

10. Consultas de verificación y análisis descriptivo

Una vez completados los procesos ETL, se ejecutan consultas de verificación sobre las tablas `personas`, `articulos`

- `SELECT * FROM personas LIMIT 5` para inspeccionar registros de personas migradas.

- `SELECT * FROM articulos LIMIT 5` para revisar artículos con cantidades y precios normalizados.

- `SELECT * FROM ventas LIMIT 5` para confirmar la correcta vinculación entre compradores y artículos.

Además, el proyecto incluye ejemplos de análisis descriptivo simple basados en consultas de agregación, tales como:

- Top 5 artículos más vendidos (suma de cantidadProductos agrupada por idArticulo y unida a la tabla articulos por idArticulo)
- Top 5 compradores con mayor número de compras (conteo de idVenta agrupado por idComprador y unido a personas por idComprador)
- Distribuciones de precios y artículos menos vendidos para explorar el comportamiento de las ventas.

Estos resultados se presentan en forma de tablas y gráficos generados con pandas y matplotlib, sirviendo como base para el análisis de datos.

11. Flujo de datos de extremo a extremo

El flujo completo de información del proyecto se puede sintetizar en las siguientes etapas:

- 1) Ingesta: los archivos JSON de personas, articulos y ventas se cargan en tres instancias de MongoDB mediante el servicio Mojolicious.
- 2) Exposición: un servicio Mojolicious expone endpoints HTTP para cada colección, devolviendo los datos en formato JSON.
- 3) Transformación: el cuaderno de Jupyter consume estos endpoints y ejecuta tres procesos ETL que limpian, normalizan y transforman los datos.
- 4) Carga: los datos procesados se insertan en la base de datos relacional SQLite (almacen.sqlite), respetando la integridad de los datos.
- 5) Análisis: sobre SQLite se ejecutan consultas SQL y se generan visualizaciones para obtener indicadores como el top 5 de artículos más vendidos.

12. Conclusiones

El Proyecto Modular – Mojo evidencia cómo integrar tecnologías de bases de datos masivas y relacionales en un mismo proyecto.

El servicio en Mojolicious actúa como capa de integración, desacoplando el origen NoSQL del destino relacional.

Finalmente, las consultas analíticas y las visualizaciones obtenidas demuestran el potencial del enfoque para generar insights.

